

Python Language Cheat Sheet

A compact study book for vocabulary, theory, syntax, practical patterns, and edge cases

What this book gives you	How to use it
Fast lookup for syntax and idioms	Read each chapter once, then use the tables as reference
Short theory explanations	Build vocabulary before memorizing syntax
Clear examples and edge cases	Run the snippets, change one thing, observe the result
Beginner-to-intermediate coverage	Return to the sections on functions, iterators, exceptions, files, OOP, typing, and packaging

Designed as clean printable study material. Code examples are intentionally short and focused so the important rule is visible.

Table of Contents

Table of Contents	2
1. Python at a Glance	5
Core vocabulary	5
2. Running Python and the REPL	5
3. Syntax Essentials	6
4. Names, Objects, Identity, and Mutability	6
5. Built-in Scalar Types	6
Numbers	7
Strings	7
6. Truthiness and Boolean Logic	7
7. Operators	8
8. Lists	8
Common list edge cases	8
9. Tuples, Ranges, and Sequence Unpacking	9
10. Dictionaries	9
11. Sets	9
12. Indexing and Slicing	10
13. Control Flow	10
if / elif / else	10
for loops	10
while loops	10
14. Comprehensions	11
15. Functions	11
Mutable default argument edge case	11
16. Scope, Closures, and Lambda	11
17. Exceptions	12

18. Context Managers and with	12
19. Files and Paths	13
20. Modules, Packages, and Imports	13
21. Classes and Objects	14
Class attribute edge case	14
22. Dataclasses	14
23. Iterables, Iterators, and Generators	14
24. Functional Tools	15
25. Decorators	15
26. Type Hints	16
27. Standard Library Essentials	16
28. JSON, CSV, and Data Serialization	16
29. Regular Expressions	17
30. Logging	17
31. Command-line Programs	18
32. Testing	18
33. Debugging	18
Reading tracebacks	19
34. Virtual Environments and Packaging Basics	19
35. Concurrency Overview	19
36. Common Built-in Functions	20
37. Common Idioms	20
38. Edge Case Checklist	21
39. Mini Reference: Syntax Patterns	21
40. Learning Path	22
Appendix A. Glossary	22
Appendix B. Quick Commands	22

Appendix C. Practice Questions

23

1. Python at a Glance

Python is a high-level, interpreted, dynamically typed language. It is designed around readability, a large standard library, and a simple object model. Everything you manipulate at runtime - integers, strings, functions, classes, modules - is an object.

Concept	Meaning	Practical consequence
Interpreted	Code is executed by the Python runtime	You usually run scripts with <code>python file.py</code> or interactively in a REPL
Dynamically typed	Names do not have fixed declared types	A variable can point to different object types over time
Strongly typed	Operations require compatible types	<code>"3" + 4</code> raises <code>TypeError</code> ; convert explicitly
Object-oriented	Data and behavior are modeled as objects	Methods are functions attached to objects
Batteries included	Large standard library	Many tasks need no external package

```
# A Python program is a sequence of statements.
name = "Ada"
age = 36
print(f"{name} is {age} years old")
```

Remember: A variable name is not a box that contains data. It is a name bound to an object. This explains many Python edge cases, especially with mutable objects.

Core vocabulary

Term	Meaning
expression	Code that produces a value: <code>2 + 3</code> , <code>len(name)</code> , <code>x > 0</code>
statement	Code that performs an action: assignment, <code>if</code> , <code>for</code> , <code>import</code> , <code>def</code>
object	A runtime value with identity, type, and data
name / variable	A label bound to an object
literal	A value written directly in code: <code>42</code> , <code>"text"</code> , <code>[1, 2]</code>
callable	An object that can be called with parentheses: <code>f()</code> , <code>len()</code> , <code>MyClass()</code>
iterable	An object that can be looped over: list, tuple, string, dict, file
iterator	Object that produces values one at a time using <code>next()</code>

2. Running Python and the REPL

The REPL is the interactive prompt. It is excellent for experiments. Scripts are better for repeatable programs. Modules are files that can be imported. Packages are directories of modules.

```
# Common commands
python --version
python script.py
python -m pip install requests
python -m venv .venv

# Activate a venv on Linux/macOS
source .venv/bin/activate

# Activate a venv on Windows PowerShell
.venv\Scripts\Activate.ps1
```

Command	Use
<code>python file.py</code>	Run a script file
<code>python -m module</code>	Run a module as a script; recommended for <code>pip</code> , <code>pytest</code> , <code>http.server</code>
<code>python -i file.py</code>	Run a script then enter interactive mode

Command	Use
help(obj)	Show documentation in the REPL
dir(obj)	List attributes available on an object
type(obj)	Show the object type

3. Syntax Essentials

Python uses indentation to define blocks. A block is a group of statements controlled by another statement, such as if, for, while, def, class, try, or with.

```
if temperature > 30:
    print("hot")      # inside the if block
    print("drink water")
print("done")        # outside the if block
```

Syntax rule	Example	Meaning
Colon starts a block	if x:	The following indented lines belong to it
Indentation matters	print(x)	Use consistent spaces; 4 spaces is standard
Comments start with #	# note	Ignored by Python
Line continuation	total = (a + b + c)	Parentheses allow clean multi-line expressions
Semicolons are possible	x = 1; y = 2	Valid but not idiomatic

Remember: IndentationError usually means inconsistent spaces/tabs or a missing indented block after a colon.

4. Names, Objects, Identity, and Mutability

Python assignment binds a name to an object. It does not copy the object unless you explicitly request a copy.

```
a = [1, 2]
b = a      # b points to the same list object
a.append(3)
print(b)   # [1, 2, 3]

c = a.copy() # shallow copy
c.append(4)
print(a)    # [1, 2, 3]
print(c)    # [1, 2, 3, 4]
```

Concept	Use	Example
identity	Which object is this?	id(x), x is y
equality	Do values compare equal?	x == y
mutable	Can be changed in place	list, dict, set
immutable	Cannot be changed in place	int, float, str, tuple, frozenset
rebinding	Make a name point elsewhere	x = x + 1

```
x = 1000
y = 1000
print(x == y) # True: same value
print(x is y) # Do not rely on this for numbers/strings

# Use "is" for identity checks, especially None:
if value is None:
    print("missing")
```

5. Built-in Scalar Types

Numbers

Type	Example	Notes
int	42, -7, 10_000	Arbitrary precision; no overflow like fixed-width C integers
float	3.14, 1e-6	Double precision; can have rounding surprises
complex	2 + 3j	Useful in mathematics and signal processing
bool	True, False	Subclass of int; True behaves like 1, False like 0

```
print(0.1 + 0.2)          # 0.30000000000000004
print(round(0.1 + 0.2, 2)) # 0.3

# For money or exact decimal arithmetic:
from decimal import Decimal
print(Decimal("0.1") + Decimal("0.2")) # 0.3
```

Remember: Never compare floats for exact equality when a tolerance is more appropriate. Use `math.isclose(a, b)`.

Strings

Strings are immutable sequences of Unicode characters. They support indexing, slicing, methods, and formatting.

```
s = "Python"
print(s[0])      # P
print(s[-1])     # n
print(s[1:4])    # yth
print(s.upper()) # PYTHON

name = "Ada"
score = 97.25
print(f"{name}: {score:.1f}%") # Ada: 97.2%
```

Pattern	Meaning
<code>s.strip()</code>	Remove surrounding whitespace
<code>s.lower()</code> , <code>s.upper()</code>	Normalize case
<code>s.split(",")</code>	Split into a list
<code>"-".join(items)</code>	Join strings with separator
<code>s.replace(old, new)</code>	Return a changed copy
<code>s.startswith(prefix)</code>	Boolean prefix test
<code>s.find(text)</code>	Index or -1 if not found
text in s	Membership test

6. Truthiness and Boolean Logic

Objects can be tested in boolean contexts. Empty containers and zero-like values are false; most other values are true.

False values	True values
False, None, 0, 0.0, "", [], {}, set(), range(0)	Non-empty strings/containers, non-zero numbers, most objects

```
items = []
if not items:
    print("no items")

name = "George"
if name and name.startswith("G"):
    print("valid")
```

Remember: and and or return one of their operands, not necessarily True or False. This enables idioms, but can surprise beginners.

```
print("hello" or "fallback") # hello
print("" or "fallback")     # fallback
print(0 and expensive_call()) # expensive_call is not executed
```

7. Operators

Category	Operators	Notes
Arithmetic	+, -, *, /, //, %, **	/ always returns float; // is floor division
Comparison	==, !=, <, <=, >, >=	Can be chained: 0 < x < 10
Boolean	and, or, not	Short-circuit evaluation
Identity	is, is not	Use for None or same-object checks
Membership	in, not in	Works with sequences, sets, dict keys
Bitwise	&, , ^, ~, <<, >>	Common in masks and low-level work
Assignment expression	:=	Assign while evaluating an expression

```
x = 7
print(x / 2)    # 3.5
print(x // 2)   # 3
print(x % 2)    # 1

if 0 < x < 10:
    print("single chained comparison")

while (line := input("name> ")) != "quit":
    print(line.upper())
```

8. Lists

A list is an ordered, mutable collection. It is the default container for sequences you want to change.

```
nums = [3, 1, 4]
nums.append(1)
nums.extend([5, 9])
nums[0] = 99
print(nums)          # [99, 1, 4, 1, 5, 9]
print(nums[1:4])     # [1, 4, 1]
print(len(nums), sum(nums))
```

Method	Effect
append(x)	Add one item at the end
extend(iterable)	Add many items
insert(i, x)	Insert before index i
pop([i])	Remove and return item, last by default
remove(x)	Remove first equal value
sort()	Sort in place
sorted(list)	Return a new sorted list
copy()	Shallow copy

Common list edge cases

```
# Dangerous: repeated references to the same inner list
bad = [[0] * 3] * 2
bad[0][0] = 99
print(bad) # [[99, 0, 0], [99, 0, 0]]

# Correct: create a new inner list each time
good = [[0] * 3 for _ in range(2)]
good[0][0] = 99
print(good) # [[99, 0, 0], [0, 0, 0]]
```


9. Tuples, Ranges, and Sequence Unpacking

A tuple is an ordered immutable collection. Tuples are often used for fixed records or returning multiple values.

```
point = (10, 20)
x, y = point
print(x, y)

first, *middle, last = [1, 2, 3, 4, 5]
print(first)    # 1
print(middle)   # [2, 3, 4]
print(last)     # 5
```

Syntax	Meaning
<code>a, b = b, a</code>	Tuple packing/unpacking; swap values
<code>x, = single_tuple</code>	Unpack a one-item tuple
<code>(1,)</code>	One-item tuple; comma is required
<code>range(5)</code>	0, 1, 2, 3, 4
<code>range(2, 10, 2)</code>	2, 4, 6, 8

10. Dictionaries

A dictionary maps keys to values. Keys must be hashable, usually immutable values such as strings, numbers, or tuples of immutable values.

```
person = {"name": "Ada", "age": 36}
print(person["name"])
print(person.get("city", "unknown"))

person["city"] = "London"
for key, value in person.items():
    print(key, value)
```

Operation	Example	Notes
Lookup	<code>d[key]</code>	Raises <code>KeyError</code> if missing
Safe lookup	<code>d.get(key, default)</code>	Returns default if missing
Insert/update	<code>d[key] = value</code>	Adds or replaces
Delete	<code>del d[key]</code>	Raises <code>KeyError</code> if missing
Iterate keys	<code>for k in d:</code>	Default iteration is over keys
Iterate pairs	<code>for k, v in d.items():</code>	Most common pattern
Default on missing	<code>setdefault, defaultdict</code>	Useful for grouping

```
# Count occurrences
counts = {}
for word in ["a", "b", "a"]:
    counts[word] = counts.get(word, 0) + 1
print(counts) # {'a': 2, 'b': 1}
```

11. Sets

A set is an unordered collection of unique hashable elements. Sets are excellent for membership tests and mathematical set operations.

```
a = {1, 2, 3}
b = {3, 4}
print(a | b) # union: {1, 2, 3, 4}
print(a & b) # intersection: {3}
print(a - b) # difference: {1, 2}
print(a ^ b) # symmetric difference: {1, 2, 4}
```

Task	Use
Remove duplicates	<code>unique = set(items)</code>
Fast membership	<code>if user_id in allowed_ids:</code>
Common values	<code>a & b</code>
Values in one but not another	<code>a - b</code>
Immutable set	<code>frozenset(items)</code>

12. Indexing and Slicing

Sequences use zero-based indexing. Negative indexes count from the end. Slicing returns a new sequence of the same general type for strings, lists, and tuples.

```
s = "abcdef"
print(s[0])      # a
print(s[-1])     # f
print(s[1:4])    # bcd
print(s[:3])     # abc
print(s[3:])     # def
print(s[:2])     # ace
print(s[::-1])   # fedcba
```

Slice	Meaning
<code>seq[start:stop]</code>	From start inclusive to stop exclusive
<code>seq[:stop]</code>	From beginning to stop exclusive
<code>seq[start:]</code>	From start to end
<code>seq[start:stop:step]</code>	Move by step
<code>seq[::-1]</code>	Reverse copy

Remember: The stop index is excluded. This makes `len(seq[a:b])` equal to `b - a` when indexes are in range.

13. Control Flow

if / elif / else

```
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
else:
    grade = "C or below"
```

for loops

```
names = ["Ada", "Grace", "Linus"]
for index, name in enumerate(names, start=1):
    print(index, name)

for a, b in zip([1, 2, 3], [10, 20, 30]):
    print(a + b)
```

while loops

```
attempts = 0
while attempts < 3:
    attempts += 1
    print("try", attempts)
else:
    print("loop finished without break")
```

Keyword	Meaning
<code>break</code>	Exit the nearest loop immediately

Keyword	Meaning
continue	Skip to the next iteration
else on loop	Runs only if the loop did not break
pass	Do nothing; placeholder statement

14. Comprehensions

Comprehensions create collections from iterables in a compact form. Use them when the transformation is simple and readable.

```
squares = [x * x for x in range(6)]
even_squares = [x * x for x in range(10) if x % 2 == 0]
name_lengths = {name: len(name) for name in ["Ada", "Grace"]}
unique_first_letters = {name[0] for name in ["Ada", "Alan", "Grace"]}

# Generator expression: lazy, does not build a list
result = sum(x * x for x in range(1_000_000))
```

Remember: Avoid complex nested comprehensions when a normal for loop is clearer. Readability is a feature in Python.

15. Functions

A function packages behavior behind a name. It can receive arguments and return a value. Without an explicit return, a function returns None.

```
def greet(name, punctuation="!"):
    return f"Hello, {name}{punctuation}"

print(greet("Ada"))
print(greet("Ada", "?"))
```

Parameter kind	Example	Meaning
positional	f(a, b)	Matched by position
keyword	f(name="Ada")	Matched by name
default	def f(x=0):	Used if caller omits the argument
var positional	*args	Collect extra positional args as tuple
var keyword	**kwargs	Collect extra keyword args as dict
keyword-only	def f(*, debug=False):	Must be passed by name
positional-only	def f(x, /):	Cannot be passed by keyword

Mutable default argument edge case

```
# Bad: the same list is reused between calls
def add_item_bad(item, items=[]):
    items.append(item)
    return items

print(add_item_bad("a")) # ['a']
print(add_item_bad("b")) # ['a', 'b'] surprise

# Good: use None as sentinel
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items
```

16. Scope, Closures, and Lambda

Python resolves names using the LEGB rule: Local, Enclosing, Global, Built-in. Assignment inside a function normally creates a local name unless declared global or nonlocal.

```
x = "global"

def outer():
    x = "enclosing"
    def inner():
        return x
    return inner

f = outer()
print(f()) # enclosing

def make_counter():
    count = 0
    def inc():
        nonlocal count
        count += 1
        return count
    return inc

counter = make_counter()
print(counter()) # 1
print(counter()) # 2
```

Remember: lambda creates a small anonymous function. Use def when the function needs a name, documentation, or multiple statements.

```
people = ["bob", "Alice", "dave"]
print(sorted(people, key=lambda s: s.lower()))
```

17. Exceptions

Exceptions represent abnormal conditions. Raise exceptions when a function cannot do what its contract promises. Catch exceptions only when you can handle them meaningfully.

```
try:
    value = int(text)
except ValueError:
    print("not a valid integer")
else:
    print("conversion succeeded", value)
finally:
    print("cleanup always runs")
```

Exception	Typical cause
SyntaxError	Invalid Python syntax
NameError	Name not defined
TypeError	Operation applied to wrong type
ValueError	Right type, invalid value
KeyError	Missing dictionary key
IndexError	Missing sequence index
FileNotFoundError	Path does not exist
PermissionError	OS denied access

```
def parse_age(text):
    age = int(text)
    if age < 0:
        raise ValueError("age cannot be negative")
    return age
```

18. Context Managers and with

A context manager defines setup and cleanup around a block. The with statement is most commonly used for files, locks, database connections, and temporary resources.

```
with open("data.txt", "w", encoding="utf-8") as f:
    f.write("hello\n")
# File is closed automatically here, even if an exception occurred.

from contextlib import contextmanager

@contextmanager
def section(name):
    print("start", name)
    try:
        yield
    finally:
        print("end", name)

with section("demo"):
    print("inside")
```

19. Files and Paths

Use pathlib for path manipulation. It is clearer and more portable than manual string concatenation.

```
from pathlib import Path

base = Path.home() / "projects" / "demo"
file_path = base / "notes.txt"

base.mkdir(parents=True, exist_ok=True)
file_path.write_text("hello\n", encoding="utf-8")
print(file_path.read_text(encoding="utf-8"))
```

Task	Preferred pattern
Join paths	<code>Path("folder") / "file.txt"</code>
Read text	<code>Path("file.txt").read_text(encoding="utf-8")</code>
Write text	<code>Path("file.txt").write_text(text, encoding="utf-8")</code>
Check exists	<code>path.exists()</code>
List directory	<code>for p in path.iterdir():</code>
Recursive search	<code>path.rglob("*.py")</code>
Open streaming file	<code>with path.open("r", encoding="utf-8") as f:</code>

20. Modules, Packages, and Imports

A module is a Python file. A package is a directory containing modules, often with an `__init__.py` file. Imports execute top-level module code once and then cache the module.

```
# math_tools.py
def add(a, b):
    return a + b

# main.py
import math_tools
print(math_tools.add(2, 3))

from math_tools import add
print(add(2, 3))
```

Import form	When to use
<code>import module</code>	Best default; keeps namespace clear
<code>import module as alias</code>	Common for long or conventional names
<code>from module import name</code>	Good for a few specific names
<code>from module import *</code>	Avoid except in rare interactive cases

```
# This block runs only when the file is executed directly,
# not when imported as a module.
if __name__ == "__main__":
    print("script entry point")
```

21. Classes and Objects

A class defines a new type. Instances are objects created from the class. Methods are functions stored on the class and receive the instance as the first argument, conventionally named `self`.

```
class Dog:
    species = "Canis familiaris" # class attribute

    def __init__(self, name):
        self.name = name # instance attribute

    def speak(self):
        return f"{self.name} says woof"

zizou = Dog("Zizou")
print(zizou.speak())
```

Term	Meaning
class	Blueprint/type definition
instance	Object created from a class
attribute	Data attached to object or class
method	Function accessed through an object/class
self	The current instance inside methods
<code>__init__</code>	Initializer called after instance creation

Class attribute edge case

```
class Bag:
    items = [] # shared by all instances: often a bug

class BetterBag:
    def __init__(self):
        self.items = [] # separate per instance
```

22. Dataclasses

Dataclasses reduce boilerplate for classes mainly used to store data. They automatically generate methods such as `__init__`, `__repr__`, and equality comparisons.

```
from dataclasses import dataclass, field

@dataclass
class User:
    name: str
    roles: list[str] = field(default_factory=list)

u = User("Ada")
u.roles.append("admin")
print(u) # User(name='Ada', roles=['admin'])
```

Remember: Use `default_factory` for mutable dataclass fields. Do not write `roles: list[str] = []`.

23. Iterables, Iterators, and Generators

An iterable can produce an iterator. An iterator produces values one at a time. A generator function uses `yield` to produce a lazy stream of values.

```

items = [10, 20, 30]
it = iter(items)
print(next(it)) # 10
print(next(it)) # 20

for x in items:
    print(x)      # for uses iteration protocol internally

def countdown(n):
    while n > 0:
        yield n
        n -= 1

for value in countdown(3):
    print(value) # 3, 2, 1

```

Object	Description
iterable	Can be passed to iter(); usable in for loops
iterator	Has __next__(); remembers progress
generator	Iterator created by a generator function or expression
lazy evaluation	Values are produced only when requested

24. Functional Tools

Python supports a functional style where functions are values. The most useful tools are often simple: sorted with key, any, all, map/filter in limited cases, and functools utilities.

```

users = [
    {"name": "Ada", "age": 36},
    {"name": "Grace", "age": 85},
]

print(sorted(users, key=lambda u: u["age"]))
print(any(u["age"] > 80 for u in users))
print(all("name" in u for u in users))

```

Tool	Meaning
map(func, iterable)	Apply function lazily to each item
filter(func, iterable)	Keep items where function returns true
any(iterable)	True if at least one element is truthy
all(iterable)	True if all elements are truthy
zip(a, b)	Pair elements from iterables
enumerate(iterable)	Pair index and value
functools.lru_cache	Memoize function results

25. Decorators

A decorator is a function that receives a function and returns a replacement function. It is written with @decorator_name above a function definition.

```

from functools import wraps

def log_calls(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print("calling", func.__name__)
        return func(*args, **kwargs)
    return wrapper

@log_calls
def add(a, b):
    return a + b

```

```
print(add(2, 3))
```

Remember: `@decorator` is syntax sugar for `function = decorator(function)`. Use `functools.wraps` so the wrapped function keeps its name and documentation metadata.

26. Type Hints

Type hints document intended types and allow static type checkers to detect many mistakes before runtime. Python itself usually does not enforce hints at runtime.

```
def repeat(text: str, times: int = 1) -> str:
    return text * times

names: list[str] = ["Ada", "Grace"]
scores: dict[str, int] = {"Ada": 95}

value: int | None = None
```

Hint	Meaning
<code>list[str]</code>	List of strings
<code>dict[str, int]</code>	Dictionary from string keys to integer values
<code>tuple[int, str]</code>	Two-item tuple: int then str
<code>set[int]</code>	Set of integers
<code>int None</code>	Integer or None
<code>Callable[[int], str]</code>	Function taking int and returning str
Any	Opt out of type checking for this value

27. Standard Library Essentials

Module	Use
<code>os</code>	Operating system interfaces; environment variables
<code>sys</code>	Interpreter details, <code>argv</code> , <code>exit</code>
<code>pathlib</code>	Object-oriented paths
<code>datetime</code>	Dates and times
<code>json</code>	JSON parsing and writing
<code>csv</code>	CSV files
<code>re</code>	Regular expressions
<code>subprocess</code>	Run external commands
<code>logging</code>	Application logs
<code>argparse</code>	Command-line arguments
<code>collections</code>	Counter, defaultdict, deque, namedtuple
<code>itertools</code>	Efficient iterator building blocks
<code>functools</code>	Higher-order function utilities
<code>statistics</code>	Basic statistics
<code>sqlite3</code>	Built-in SQLite database access
<code>venv</code>	Virtual environment creation
<code>unittest</code>	Built-in testing framework

28. JSON, CSV, and Data Serialization


```
import json

data = {"name": "Ada", "active": True, "score": 97.5}
text = json.dumps(data, indent=2)
print(text)

loaded = json.loads(text)
print(loaded["name"])

import csv

rows = [
    {"name": "Ada", "score": 97},
    {"name": "Grace", "score": 99},
]

with open("scores.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["name", "score"])
    writer.writeheader()
    writer.writerows(rows)
```

Remember: Always use `newline=""` when writing CSV files with the `csv` module, especially on Windows, to avoid extra blank lines.

29. Regular Expressions

Regular expressions describe text patterns. Use them when simple string methods are not enough. Keep patterns readable and test them with examples.

```
import re

text = "Order 123 costs 45.90 EUR"
match = re.search(r"Order (\d+) costs (\d+\.\d{2})", text)
if match:
    order_id = match.group(1)
    amount = match.group(2)
    print(order_id, amount)
```

Pattern	Meaning
.	Any character except newline
\d	Digit
\w	Word character
\s	Whitespace
*	Zero or more
+	One or more
?	Zero or one, or non-greedy modifier
{m,n}	Between m and n repetitions
(...)	Capture group
(?:...)	Non-capturing group
^ and \$	Start and end anchors

30. Logging

Use logging instead of `print` in programs, services, and libraries. Logging records severity, timestamp, module, and message. It can write to console, files, syslog, or other destinations.

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(name)s: %(message)s"
)
```

```
log = logging.getLogger(__name__)
log.info("service started")
log.warning("configuration value missing; using default")
```

Level	Use
DEBUG	Detailed diagnostic information
INFO	Normal operational messages
WARNING	Something unexpected but program continues
ERROR	An operation failed
CRITICAL	Serious failure; program may stop

31. Command-line Programs

```
import argparse

parser = argparse.ArgumentParser(description="Process a file")
parser.add_argument("path", help="input file path")
parser.add_argument("--limit", type=int, default=10)
parser.add_argument("--verbose", action="store_true")
args = parser.parse_args()

print(args.path, args.limit, args.verbose)
```

Pattern	Meaning
positional argument	Required value by position
optional argument	Flag such as --limit 10
type=int	Convert input string to int
default=10	Value used when flag omitted
action="store_true"	Boolean flag
choices=[...]	Restrict allowed values

32. Testing

Tests are executable examples that protect behavior from accidental changes. `pytest` is popular, but the standard library also includes `unittest`.

```
# file: math_tools.py
def add(a, b):
    return a + b

# file: test_math_tools.py
def test_add():
    assert add(2, 3) == 5
    assert add(-1, 1) == 0

# Standard library unittest style
import unittest

class TestMath(unittest.TestCase):
    def test_add(self):
        self.assertEqual(add(2, 3), 5)

if __name__ == "__main__":
    unittest.main()
```

Remember: Good tests include normal cases, edge cases, and error cases. Edge cases often reveal incorrect assumptions.

33. Debugging

Tool	Use
print()	Fast temporary inspection
logging	Persistent diagnostics
breakpoint()	Start debugger at this line
traceback	Error stack showing call chain
repr(obj)	Unambiguous representation
type(obj)	Check runtime type
dir(obj)	Explore attributes

```
def divide(a, b):
    breakpoint() # inspect a and b here
    return a / b

print(divide(10, 2))
```

Reading tracebacks

A traceback shows the call chain. Read from bottom to find the exception type and message, then move upward to find the line in your code that caused it.

34. Virtual Environments and Packaging Basics

A virtual environment isolates dependencies for a project. This prevents one project from breaking another by requiring incompatible package versions.

```
python -m venv .venv
source .venv/bin/activate          # Linux/macOS
.venv\Scripts\Activate.ps1        # Windows PowerShell
python -m pip install package_name
python -m pip freeze > requirements.txt
python -m pip install -r requirements.txt
```

File	Purpose
requirements.txt	Pinned or listed dependencies for simple projects
pyproject.toml	Modern project metadata and build configuration
README.md	Human-readable project explanation
src/	Common location for package source code
tests/	Test files

35. Concurrency Overview

Python has several concurrency models. Choose based on the type of work: I/O-bound, CPU-bound, or event-driven.

Tool	Best for	Notes
threading	Blocking I/O tasks	Shared memory; beware race conditions
multiprocessing	CPU-bound parallel work	Separate processes; higher overhead
asyncio	Many concurrent I/O operations	Uses async/await and event loop
concurrent.futures	Simple thread/process pools	High-level interface

```
import asyncio

async def fetch_fake(name):
    await asyncio.sleep(1)
    return f"done {name}"

async def main():
    results = await asyncio.gather(fetch_fake("a"), fetch_fake("b"))
```

```
print(results)

asyncio.run(main())
```

Remember: async does not automatically make CPU-heavy code faster. It helps when tasks spend time waiting for I/O.

36. Common Built-in Functions

Function	Use
len(x)	Number of items
sum(nums)	Sum numeric values
min(x), max(x)	Smallest/largest item
sorted(x, key=...)	Return sorted list
reversed(x)	Reverse iterator
enumerate(x, start=0)	Index and item pairs
zip(a, b)	Pair values from iterables
range(n)	Lazy integer sequence
isinstance(x, T)	Runtime type check
hasattr(obj, name)	Attribute existence check
getattr(obj, name, default)	Dynamic attribute lookup
open(path, mode)	Open file stream
round(x, ndigits)	Rounded number
abs(x)	Absolute value
all(x), any(x)	Boolean aggregation

37. Common Idioms

```
# Swap values
a, b = b, a

# Default when value may be None
name = provided_name if provided_name is not None else "Anonymous"

# Build a dictionary from pairs
pairs = [("a", 1), ("b", 2)]
d = dict(pairs)

# Group values
from collections import defaultdict
groups = defaultdict(list)
for item in items:
    groups[item.category].append(item)

# Count values
from collections import Counter
counts = Counter(words)
```

Need	Idiomatic solution
Index while looping	enumerate(items)
Loop two lists together	zip(a, b)
Sort objects	sorted(items, key=lambda x: x.field)
Avoid missing dict keys	dict.get, defaultdict, setdefault
Open resource safely	with ... as ...
Create simple data holder	dataclass

Need	Idiomatic solution
Check for absence	is None, is not None

38. Edge Case Checklist

Area	Edge case	Safer habit
Defaults	Mutable default argument reused	Use None sentinel or default_factory
Copies	Assignment does not copy	Use copy(), deepcopy() when needed
Identity	is vs == confusion	Use is for None/same object; == for value equality
Floats	Rounding surprises	Use math.isclose or Decimal for exact decimal needs
Strings	Backslashes in paths/regex	Use raw strings for regex: r"\d+"
Dicts	Missing key	Use get or handle KeyError
Files	Encoding differs by system	Pass encoding="utf-8" explicitly
Exceptions	Bare except hides bugs	Catch specific exceptions
Loops	Modifying list while iterating	Iterate over a copy or build a new list
Dataclasses	Mutable field shared	Use field(default_factory=list)
Imports	Top-level side effects	Use if __name__ == "__main__" for script code
Comprehensions	Too much logic in one line	Prefer normal loops for readability

39. Mini Reference: Syntax Patterns

```
# Variables
x = 10
name: str = "Ada"

# Conditions
if condition:
    ...
elif other_condition:
    ...
else:
    ...

# Loops
for item in iterable:
    ...

while condition:
    ...

# Functions
def function_name(arg1, arg2="default"):
    return result

# Classes
class ClassName:
    def __init__(self, value):
        self.value = value

# Exceptions
try:
    risky()
except SpecificError as exc:
    handle(exc)
finally:
    cleanup()

# Context manager
with open("file.txt", encoding="utf-8") as f:
    text = f.read()
```

40. Learning Path

Use this path to turn the cheat sheet into practical skill. Each step builds vocabulary and confidence.

Stage	Focus	Practice project
1	Basic syntax, strings, lists, dictionaries	Text statistics script
2	Files, pathlib, JSON, CSV	Folder inventory exporter
3	Functions, exceptions, logging	Command-line configuration validator
4	Classes and dataclasses	Small task tracker
5	Testing and packaging	Reusable utility module with tests
6	APIs and automation	Fetch data, save JSON/CSV, log results
7	Concurrency basics	Async downloader or threaded file processor

Remember: The fastest way to learn Python is not to memorize everything. Learn the vocabulary, run small examples, then repeatedly build small useful tools.

Appendix A. Glossary

Term	Definition
argument	A value passed to a function call.
attribute	A value accessed through dot notation, such as <code>obj.name</code> .
call	Execute a callable object using parentheses.
class	A user-defined type.
decorator	A function that transforms another function or class.
exception	An object representing an error or abnormal condition.
expression	Code that produces a value.
function	Reusable block of code that can receive inputs and return output.
hashable	Object that can be used as a set element or dictionary key.
immutable	Cannot be changed in place.
instance	An object created from a class.
iterable	Object that can be looped over.
iterator	Object that returns next values and remembers state.
method	Function associated with an object or class.
module	A Python file that can be imported.
mutable	Can be changed in place.
package	A collection of modules.
parameter	A name in a function definition that receives an argument.
scope	Region where a name can be resolved.
statement	Code that performs an action.
truthy/falsy	How an object behaves in boolean context.
type hint	Annotation that documents expected type.
virtual environment	Isolated Python environment for dependencies.

Appendix B. Quick Commands

```
# Run
python script.py
python -m module
```

```
# Environment
python -m venv .venv
python -m pip install package
python -m pip list
python -m pip freeze > requirements.txt

# Formatting and checks (external tools, if installed)
python -m black .
python -m ruff check .
python -m mypy .
python -m pytest

# Simple local web server
python -m http.server 8000
```

Appendix C. Practice Questions

Question	What to practice
Why does b change when a.append() is called after b = a?	Names, references, mutability
Why should mutable default arguments be avoided?	Function object lifetime and defaults
When should you use is instead of ==?	Identity vs equality
What is the difference between a list comprehension and a generator expression?	Memory and lazy evaluation
Why does open(..., encoding="utf-8") matter?	Portability and text encodings
What does a traceback tell you?	Debugging call stacks
When is a dataclass better than a normal class?	Data modeling and boilerplate
Why should exceptions be specific?	Correct error handling
What is the difference between import module and from module import name?	Namespaces
What kind of work benefits from asyncio?	Concurrency model selection